Dockerized Node.js App on AWS EC2 - FULL CODE GUIDE (V2)

This version has 100% visible code blocks (light background) + separate files

PROJECT STRUCTURE

```
nodejs-docker-app/
■■■ app.js
■■■ package.json
■■■ Dockerfile
■■■ .dockerignore
```

FILE 1: app.js - COPY THIS EXACT CODE

```
const http = require('http');

const hostname = '0.0.0.0';
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/html');
  res.end(`
    <h1>Hello from Node.js inside Docker! ■</h1>
    <p>This app is running in a container on AWS EC2.</p>
    <p>Time: ${new Date().toISOString()}</p>
    <p>Path: ${req.url}</p>
  `);
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

FIXED: Used backticks ` ` for console.log so ${hostname} and ${port} will print correctly. Old PDF had single quotes.

FILE 2: package.json - COPY THIS EXACT CODE

```
{
  "name": "nodejs-docker-app",
  "version": "1.0.0",
  "description": "Node.js app dockerized for EC2",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "keywords": ["nodejs", "docker", "aws"],
  "author": "",
  "license": "MIT"
}
```

FILE 3: Dockerfile - COPY THIS EXACT CODE

```
# Use official lightweight Node.js image
FROM node:20-alpine

# Set working directory inside container
```

```
WORKDIR /usr/src/app

# Copy package files first for better caching
COPY package*.json ./

# Install only production deps
RUN npm install --only=production

# Copy rest of the app files
COPY . .

# App listens on port 3000
EXPOSE 3000

# Run the app
CMD ["npm", "start"]
```

FILE 4: .dockerignore - COPY THIS EXACT CODE

```
node_modules
npm-debug.log
.git
.gitignore
README.md
.env
Dockerfile
.dockerignore
```

STEP-BY-STEP COMMANDS

Step 1: Build Image

```
docker build -t my-node-app:1.0 .
```

Step 2: Run & Test Locally

```
docker run -d -p 8080:3000 --name node-test my-node-app:1.0
# Now open: http://localhost:8080
docker logs node-test
docker stop node-test
docker rm node-test
```

Step 3: Push to Docker Hub

```
docker login
docker tag my-node-app:1.0 YOUR_DOCKERHUB_USERNAME/my-node-app:1.0
docker push YOUR_DOCKERHUB_USERNAME/my-node-app:1.0
```

Step 4: Launch EC2 + Open Port 3000

AWS Console -> EC2 -> Launch Instance -> Amazon Linux 2023 -> t2.micro -> Create Key Pair -> Security Group: Allow SSH (22) My IP + Custom TCP 3000 Anywhere 0.0.0.0/0

Step 5: Install Docker on EC2 (Amazon Linux)

```
chmod 400 your-key.pem
ssh -i your-key.pem ec2-user@YOUR_EC2_IP

sudo yum update -y
sudo yum install -y docker
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user

exit
# SSH again
ssh -i your-key.pem ec2-user@YOUR_EC2_IP
```

Step 6: Run on EC2

```
docker pull YOUR_DOCKERHUB_USERNAME/my-node-app:1.0
docker run -d -p 3000:3000 --restart unless-stopped --name node-live YOUR_DOCKERHUB_USERNAME/my-node
docker ps
docker logs node-live
```

Step 7: Open in Browser

```
http://YOUR_EC2_PUBLIC_IP:3000
```

Done! Your site is live to the world. ■